

Technical Report
BC Wild Watch - Campus
Animal Safety Reporting App
May 2026
INL37(8)1
Group 3



Table of Contents

1. Introduction	3
1.1 Background and Context	3
1.2 Problem Statement	3
2. Project Scope and Objectives	3
2.1 Scope.....	3
2.2 Objectives	3
3. System Architecture	3
3.1 Architecture Overview	3
3.2. Presentation Layer	3
3.3 Business Logic and Process Layer	4
3.4 Data Layer	4
3.5 Analytics and Intelligence Layer	4
4. System Functionality	4
4.1 Incident Reporting Workflow	4
4.2 Chatbot Integration	4
4.3 Automated Alerts	5
4.3.1 Power Automate enables:	5
4.3.2 Benefits:	5
5. Deployment Strategy	5
5.1 Deployment Approach	5
5.2 A QR code will be generated to:	5
6. Risk Assessment.....	6
7. Architecture Diagram	6
8. Leadership and Team Coordination	7
8.1 Team Roles	7
8.2 RACI MATRIX:.....	7

1. Introduction

1.1 Background and Context

Snakes, bees, ants and stray animals are among the hazardous or bothersome species that frequently pose as a threat to the safety of Belgium Campus. Staff and students may be harmed or distressed by these situations. BC Wild Watch was designed as a digital campus safety solution that allows for real-time reporting and tracking of animal sightings in order to address this problem. This project, which started out as a prototype, aims to produce a Minimum Viable Product (MVP) that is secure, scalable and deployable for practical application.

1.2 Problem Statement

The project's goal is to create a scalable, secure, and web-accessible system that lets students and staff record animal sightings and enables campus security to react quickly with real-time alerts and statistics.

2. Project Scope and Objectives

2.1 Scope

The BC Wild Watch™ system is designed to:

- Allow students and staff to report animal sightings via a mobile/web application
- Notify campus security in real time
- Track and manage incidents
- Provide safety guidance via a chatbot
- Deliver analytics through dashboards

2.2 Objectives

- Develop a functional MVP using Microsoft Power Platform
- Enable real-time alerting and tracking
- Ensure secure data storage and access
- Provide actionable insights using data analytics
- Support decision-making for campus safety improvement

3. System Architecture

3.1 Architecture Overview

The system follows a layered architecture approach, ensuring modularity, scalability, and maintainability.

3.2. Presentation Layer

This layer presents user interaction through Power Apps.

Key features:

- Photo capture for incident documentation
- GPS-based location detection
- Offline functionality for limited connectivity
- Guided form-based reporting
- Real-time status updates

3.3 Business Logic and Process Layer

This layer is powered by Power Automate workflows

Core Functionalities:

- **Incident Triage Workflow:** Categorises incidents based on severity and animal type
- **Notification Cascade:** Sends alerts to relevant security personnel
- **Approval Processes:** Escalates high-risk incidents
- **System Integration:** Connects to email, SMS, and external services

3.4 Data Layer

The system uses Microsoft Dataverse as a central data repository

Capabilities:

- Structured data management
- Row-level and column-level security
- Audit trails for compliance
- Scalable storage with backup and recovery

3.5 Analytics and Intelligence Layer

Powered by Power BI, this layer provides:

- Live incident monitoring dashboards
- Trend analysis of animal activity
- Safety performance metrics

4. System Functionality

4.1 Incident Reporting Workflow

1. User submits report via Power Apps
2. Data stored in Dataverse
3. Power Automate triggers alert
4. Security receives notification
5. Incident status updated in system

4.2 Chatbot Integration

The chatbot enhances user interaction by providing safety guidance.

Implementation Approach:

- Integration with OpenAI/Gemini via API

- Power Automate flow handles communication
- Responses displayed within Power Apps

Additional Functionality:

- Users can confirm incident reporting
- System triggers alerts upon confirmation

4.3 Automated Alerts

4.3.1 Power Automate enables:

- Real-time alert generation
- Inclusion of critical details (animal type, time, risk level)
- Logging alerts into Notification table

4.3.2 Benefits:

- Reduced manual effort
- Faster response times
- Improved operational efficiency

5. Deployment Strategy

5.1 Deployment Approach

The system will be deployed as a web-accessible solution using free-tier cloud platforms

Steps:

1. Develop Power App
2. Configure Dataverse database
3. Implement Power Automate workflows
4. Integrate chatbot API
5. Publish dashboards via Power BI
6. Host web access portal
7. Enable secure authentication

5.2 A QR code will be generated to:

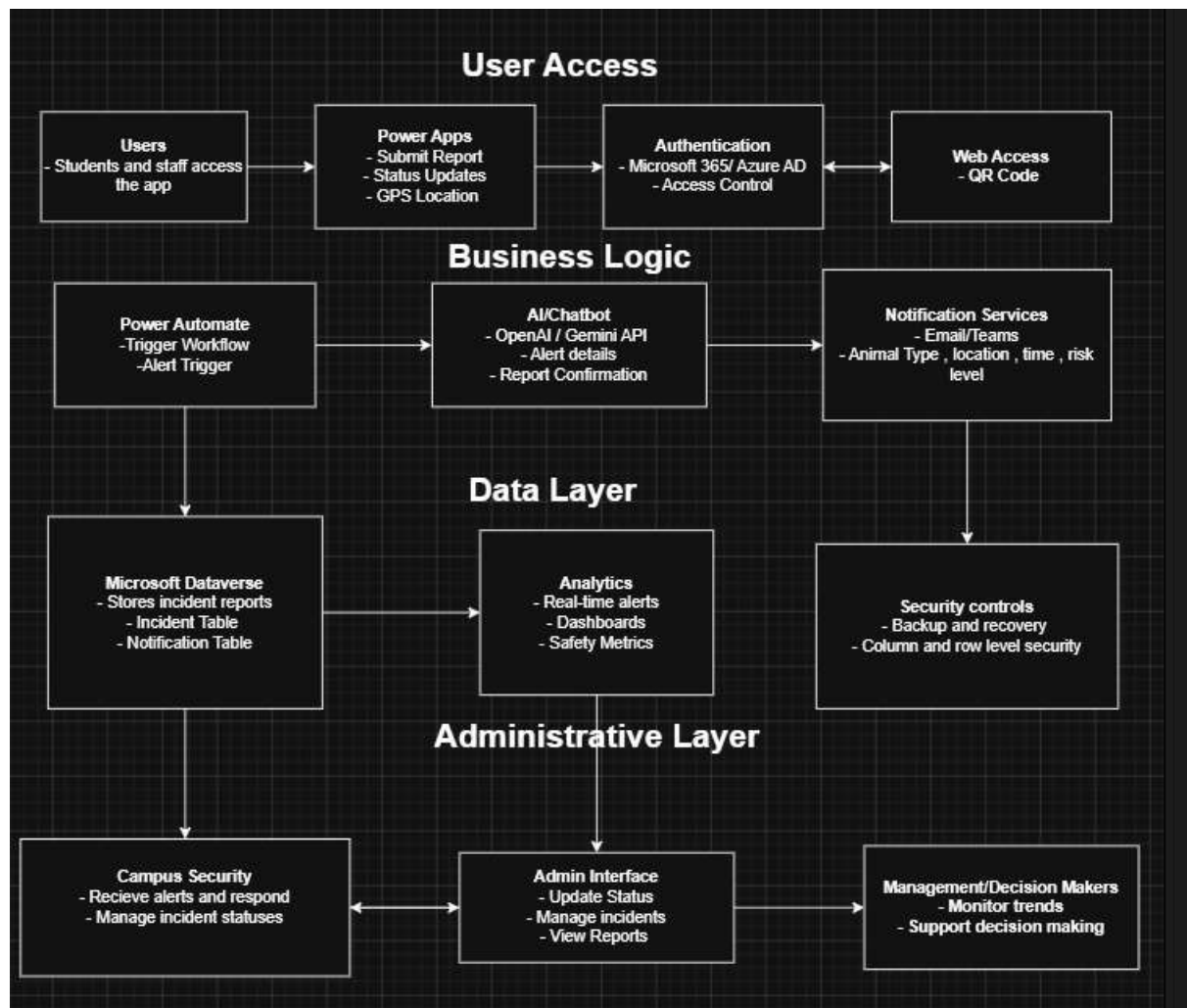
- Provide instant system access
- Support mobile usage
- Enhance demonstration and usability



6. Risk Assessment

Risk	Impact	Mitigation
System Failure	High	Reliable cloud hosting
Unauthorised access	High	Strong authentication
API failure (chatbot)	Medium	Fallback responses
Data Inconsistency	Medium	Validation rules
Connectivity	Low	Offline functionality

7. Architecture Diagram



8. Leadership and Team Coordination

8.1 Last Year's Team Roles

This is a list of all members that created the power apps and all associated functions for this project. The work was kept and improved in this year's project.

Name	Role
Rivan Martiz - Team Leader	Dashboard Development
Aiden Moses	Power Apps Developer
Ian Visser	Power Apps Developer
Jason Crous	Power Apps Developer
Nkosana Mtsweni	Power Automate
Lethebe Kutloano Maine	Technical Report
Denzel Takunda Muranganwa	Dataverse Design
Reinhardt Kleynhans	Dataverse Design
Nicholas Vronka	Dashboards
Mutshidzi Nduvheni	Chatbot
Sandile Machabane	Technical Report

8.1 Team Roles

This is a list of all members that created the power apps and all associated functions for this project. The work was kept and improved in this year's project.

Name	Role
Rivan Martiz - Team Leader	
Aiden Moses	Power Apps Front End
Ian Visser	
Nkosana Mtsweni	Power Apps Front End
Lethebe Kutloano Maine	Documentation
Denzel Takunda Muranganwa	
Nicholas Vronka	
Sandile Machabane	Documentation
Jordan Lee Naido	Chatbot Improvement

8.2 RACI MATRIX:

Tasks	Aiden	Denzel	Ian	Jordan	Lethebe	Nicolas	Nkosana	Rivan	Sandile
Power Apps Frontend	R/A	I	I	I	I	I	R	I	I
Documentation	C	C	C	C	R/A	C	C	C	R/A
Chat bot	I	I	I	R/A	I	I	I	I/A	I
Data Storage / Dashboards	I	I	I	I	I	R/A	I	R/A	I
Security/ Authentication	I/A	R/A	R/A	I	I	I	I	I	I

R – Responsible

A - Accountable

C – Consulted

I – Informed